Weekly Work Report – DevOps Intern

Week 5

Name: Kailas R Pillai

Day 1 - 12/01/2026

Tasks Completed:

- **Task1:** Learned AWS services fundamentals and basics.
- **Task2:** Learned how pricing works for above services and how services work.

Learnings:

- **EC2:**
  - Virtual servers in the cloud.
  - EC2 runs inside a VPC
  - Uses EBS for storage
  - Permissions via IAM Role
  - Pay for:
    - Instance running time
    - EBS volumes
    - Data transfer out
- **EBS:**
  - Hard disk for EC2.
  - Attached to EC2
  - Persists even if EC2 stops
  - Pay for:
    - GB per month
    - IOPS (for io1/io2 volumes)
    - Snapshots stored in S3
  - **EBS still costs** Even if EC2 is stopped
- **S3 (Simple Storage Service)**
  - Object storage (files, images, backups).
  - Buckets store Objects
  - S3 is integrated with Cloudfront for CDN
  - Pay for:
    - Storage (GB/month)
    - Requests (PUT, GET, LIST)
    - Data transfer out

- **CloudFront (CDN)**
    - Content Delivery Network.
    - Caches content at edge locations
    - Reduces latency & S3/EC2 load
    - CloudFront is integrated with S3 / ALB / EC2
    - Pay for:
        - Data transfer out
        - HTTP/HTTPS requests
- **ECS (Elastic Container Service)**
    - Run Docker containers.
    - Launch types
        - **EC2 launch type:** manage EC2
        - **Fargate:** Serverless containers (Simple but slightly expensive)
    - **Pay for**
        - EC2 mode: Pay for EC2
        - Fargate:Pay for vCPU seconds and Memory seconds
- **ECR (Elastic Container Registry)**
    - Docker image repository.
    - Pay for:
        - **Image storage (GB/month)**
        - **Data transfer out**
- **RDS (Relational Database Service)**
    - Managed databases (MySQL, Postgres, etc.)
    - Pay for:
        - DB instance hours
        - Storage
        - IOPS
        - Backups beyond free limit
- **NFS / EFS (Elastic File System)**
    - Shared file system (NFS). Files stored in EFS can be accessible to more than one instance at the same time.
    - You pay for:
        - GB stored
- **VPC**
    - Your private AWS network.
    - VPC itself is free
    - Pay for:
        - NAT Gateway
        - VPN
- **VPC Peering**
    - Connect two VPCs.
- **Load Balancer (ALB / NLB)**
    - A load balancer is a service that distributes incoming traffic across multiple servers so that no single server gets overloaded.

        - Pay for:
            - LB hours
            - LCU (Load Balancer Capacity Units)
- **Auto Scaling**
    - Automatically increase/decrease EC2
    - No extra cost
    - Pay for EC2 instances launched
- **IAM**
    - Identity & permissions.
    - Free to use
- **Secrets Manager**
    - Store secrets (DB passwords, API keys)
    - Pay for:
        - Secret per month
        - API calls
- **ACM (Certificate Manager)**
    - SSL certificates.
    - **Free** for AWS services
- **Route 53**
    - It **converts a domain name into an IP address** so users can reach your application.
    - Pay for:
        - Hosted zone per month
        - DNS queries
- **Lambda**
    - Serverless functions.
    - Pay for:
        - Number of invocations
        - Execution time (ms)
        - Memory allocated

Day 2 - 13/01/2026

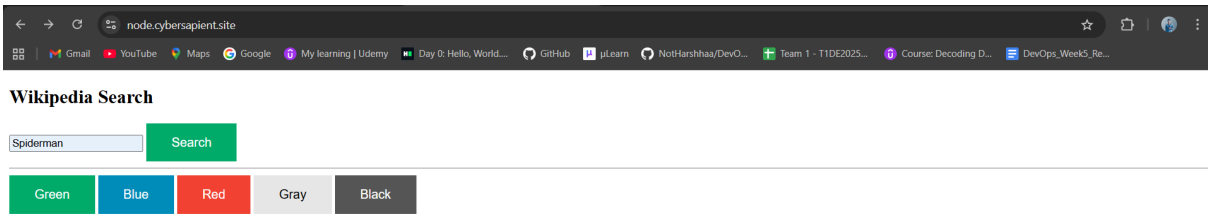
Tasks Completed:

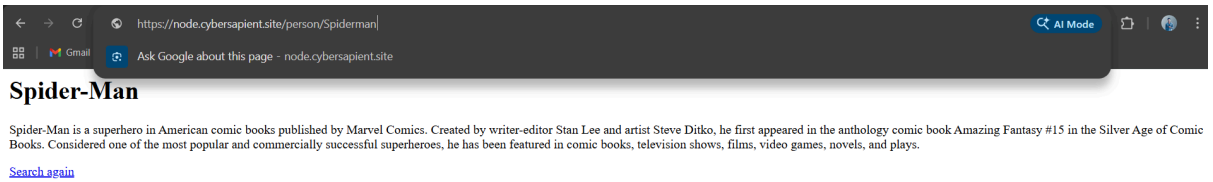
- **Task1:** Added a nodejs application to github repo, deployed it to ECS using github actions and connected it with domain using route53.
- **Task2:** Added a static website to github repo, pushed it to S3 and deployed it in cloudfront using github actions connecting it with domain using route53.

Learnings:

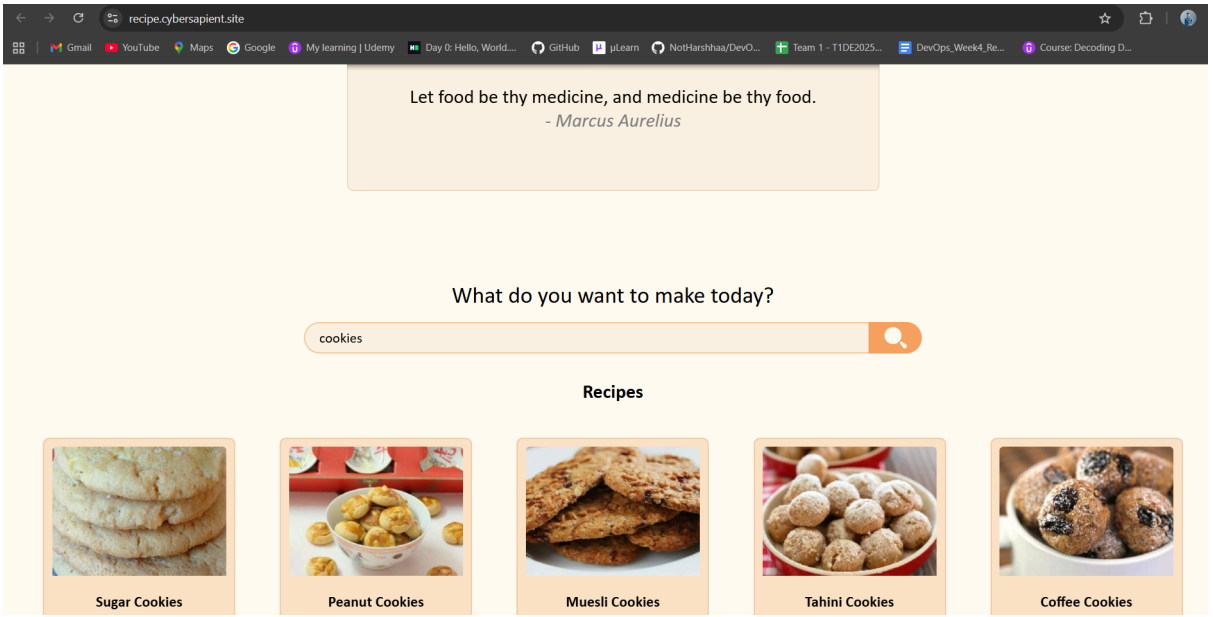
- **Deployment of Node.js Application to AWS ECS with Custom Domain**
    - Deploy a Node.js application from a GitHub repository to AWS ECS (Fargate) using GitHub Actions, expose it publicly, and connect it to a custom subdomain (node.cybersapient.site) with proper accessibility to HTTP and HTTPS and www to non-www redirection.
    - A Dockerfile was created for Node.js application.
    - The application listens on port 3000.
    - Created ECR for container storage and ECS cluster, task definition and service in AWS.
    - Allow inbound 3000 for ECS task security group.
    - Set port mapping to 3000.
    - A deploy.yml workflow was created.
    - On push to main branch:
        - Build Docker image
        - Push to ECR
        - Update ECS task definition
        - Deploy new revision to ECS service
    - GitHub Actions completed successfully and automated deployments.
    - ACM certificate created for node.cybersapient.site and www.node.cybersapient.site.
    - ALB Security Group allows inbound 80 and 443 from anywhere.
    - Internet-facing Application Load Balancer created.
    - Target group type: IP
    - Target port: 3000
    - ECS service registered tasks automatically into the target group.
    - Added HTTP : 80 Listener
    - Redirect to HTTPS
    - Added HTTPS : 443 Listener
    - Added 2 rules
    - Default rule redirects to the target group which has registered ECS task.
    - Specified the ACM certificate generated for the domains.
    - Second rule with if condition redirecting to https where host is node.cybersapient.site when host header is www.node.cybersapient.site.
    - Created records **www.node** and **node** in the created hosted zone cybersapient.site. Set alias set for ALB and set the alias target to the ALB DNS name.

- The Application was successfully accessed when visited via www.node.cybersapient.site which redirects to https://node.cybersapient.site.





- **Static Website Deployment on AWS using S3**
  - Website type: Static website (HTML, CSS, JavaScript)
  - Created S3 bucket for storing the static website
  - Bucket name: recipe.cybersapient.site
  - Region: ap-south-1
  - Enabled Block all public access to prevent direct access to S3 objects.
  - Access method: CloudFront Origin Access Control (OAC)
  - Created bucket policy which explicitly DENY public access and allow cloudfront.amazonaws.com service to read objects.
  - Issued certificate for domains recipe.cybersapient.site and www.recipe.cybersapient.site
  - CloudFront only supports ACM certificates from us-east-1
  - Created Cloudfront distribution for CDN and set origin to S3 bucket
  - Set default root object as index.html
  - Provided the SSL certificate.
  - Created github actions to automatically deploy website on every push.
  - Created records **www.recipe** and **recipe** in the created hosted zone cybersapient.site. Set alias which targets cloudfront.
  - The Application was successfully accessed via the subdomains.



Recipe Website

Day 3 - 14/01/2026

Tasks Completed:

- **Task1:** Learned how to increase EBS volume in EC2
- **Task2:** Setup pritunl for VPN in EC2
- **Task3:** Learned how to create services using AWS CLI

Learnings:

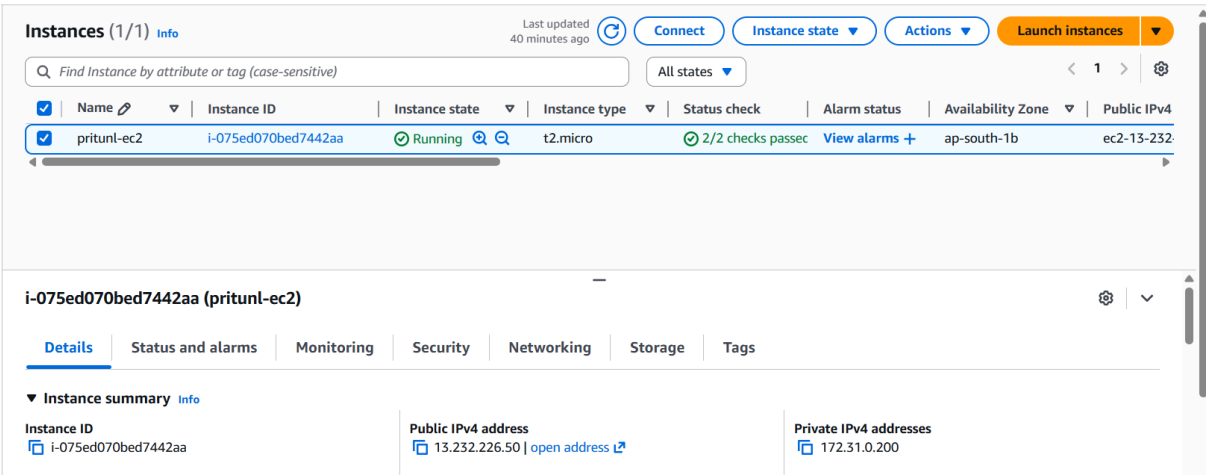
- **Increase EBS volume for EC2 instance**
  - Selected the instance and noted volume ID in storage tab
  - Logged into the instance. View current block information using lsblk.

```
ubuntu@ip-172-31-0-200:~$ lsblk
NAME     MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0      7:0    0 27.6M  1 loop /snap/amazon-ssm-agent/11797
loop1      7:1    0 73.9M  1 loop /snap/core22/2133
loop2      7:2    0 50.8M  1 loop /snap/snapd/25202
xvda     202:0    0    8G  0 disk
├─xvda1  202:1    0    7G  0 part /
├─xvda14 202:14   0    4M  0 part
├─xvda15 202:15   0  106M  0 part /boot/efi
└─xvda16 259:0    0  913M  0 part /boot
ubuntu@ip-172-31-0-200:~$
```

- Navigated to volumes and selected the volume with the specific volume ID.
- Selected Modify Volume and increased the volume.
- Logged into the EC2 and viewed block storage information using lsblk.
- Volume was found to be increased.

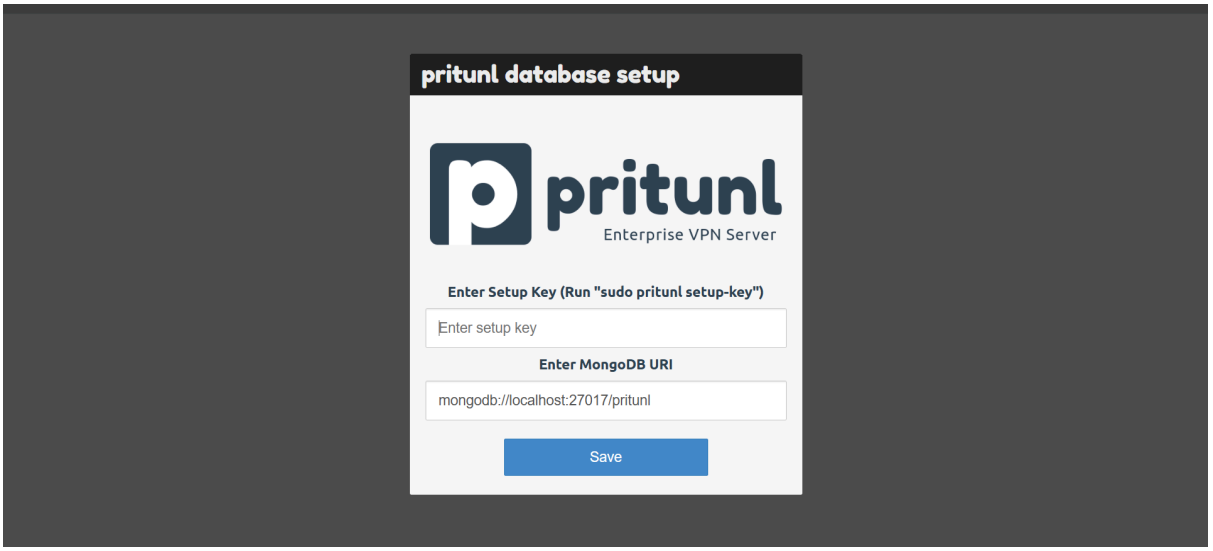
```
ubuntu@ip-172-31-0-200:~$ lsblk
NAME     MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0      7:0    0 27.6M  1 loop /snap/amazon-ssm-agent/11797
loop1      7:1    0 73.9M  1 loop /snap/core22/2133
loop2      7:2    0 50.8M  1 loop /snap/snapd/25202
xvda     202:0    0   12G  0 disk
├─xvda1  202:1    0   11G  0 part /
├─xvda14 202:14   0    4M  0 part
├─xvda15 202:15   0  106M  0 part /boot/efi
└─xvda16 259:0    0  913M  0 part /boot
ubuntu@ip-172-31-0-200:~$
```

- **Pritunl Setup**
  - The Pritunl VPN allows secure access to AWS private resources such as private EC2 instances and RDS.

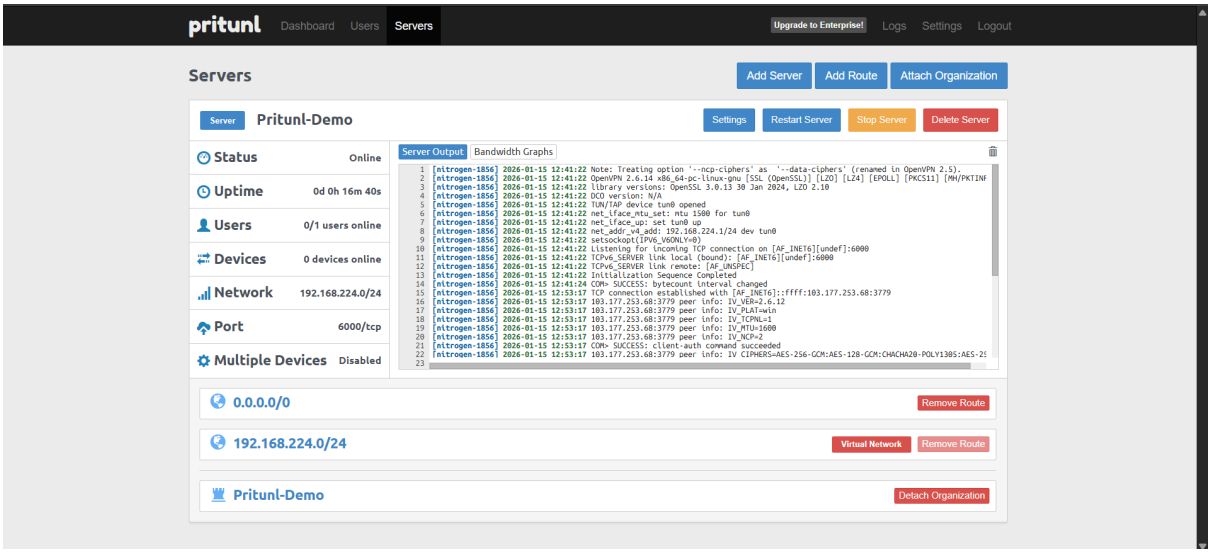


- Logged into the created instance and installed Mongodb
- Started and enabled MongoDB
- Pritunl uses MongoDB to store all user information and configurations.
- Installed and enables pritunl using systemctl.

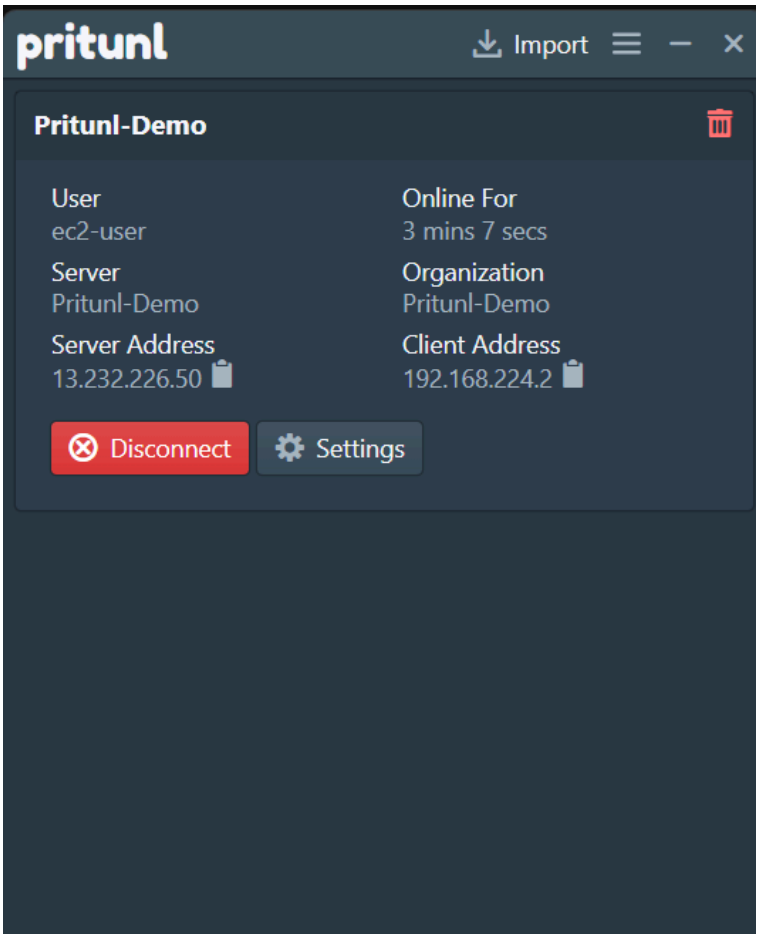
- Accessed Pritunl web UI via https://<EC2_PUBLIC_IP>
- Setup credentials for initial. Generated them using:
  - sudo pritunl setup-key
  - sudo pritunl default-password



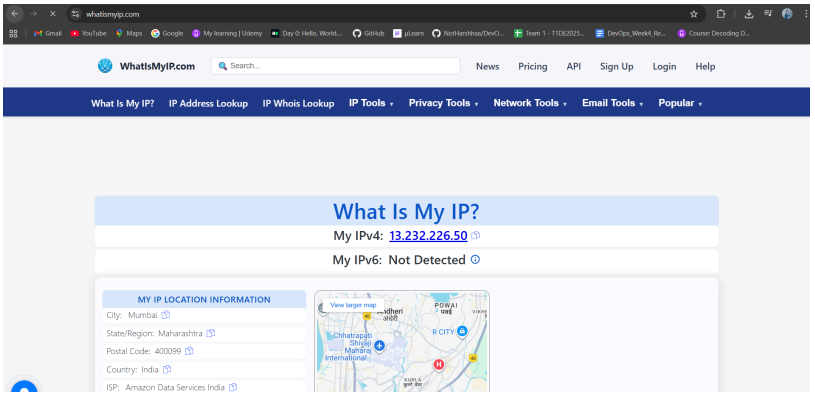
- Used the credential for initial login
- Created Organization and User
- Added VPN Server
  - Protocol: TCP
  - Port: 6000
  - Network: 10.0.0.0/24
  - DNS: 8.8.8.8
  - Security group for the instance was editted to allow ingress from 6000.
- Attached the Organization and Started the Server.



- Downloaded the VPN profile (.tar)
- Installed pritunl client and imported the VPN profile.
- Connected to the profile.



- Checked the ip of the machine.



- **Create EC2 using AWS CLI**
  - Configured AWS credentials and default region using **aws configure** for a specific profile by defining the generated secret key and access key.

- Key pairs created to securely connect to EC2 instances using **aws ec2 create-key-pair**
- Set correct file permission for the key using chmod.
- Created security group using **aws ec2 create-security-group**
- Allowed SSH and HTTP inbound rule using **aws ec2 authorize-security-group-ingress**

```
Unable to locate credentials. You can configure credentials by running "aws configure".
ubuntuuser@ubuntu25:~$ aws configure
AWS Access Key ID [None]: AKIASZ5GV2QJTNYIKDST
AWS Secret Access Key [None]: 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
Default region name [None]: ap-south-1
Default output format [None]:
ubuntuuser@ubuntu25:~$ aws ec2 create-key-pair --region ap-south-1 --key-name ec2key --query "KeyMaterail" --output text > ec2key.pem --profile kailas
ubuntuuser@ubuntu25:~$ chmod 400 ec2key.pem
ubuntuuser@ubuntu25:~$ aws ec2 create-security-group --region ap-south-1 --group-name ec2-sg --description "MyEC2 SG" --profile kailas
{
    "GroupId": "sg-0a345f5eacb9b2644",
    "SecurityGroupArn": "arn:aws:ec2:ap-south-1:193086215187:security-group/sg-0a345f5eacb9b2644"
}
ubuntuuser@ubuntu25:~$ aws ec2 authorize-security-group-ingress --group-name ec2-sg --protocol tcp --port 22 --cidr 0.0.0.0/0
{
    "Return": true,
    "SecurityGroupRules": [
        {
            "SecurityGroupRuleId": "sgr-04e8379a4d332b308",
            "GroupId": "sg-0a345f5eacb9b2644",
            "GroupOwnerId": "193086215187",
            "IsEgress": false,
            "IpProtocol": "tcp",
            "FromPort": 22,
            "ToPort": 22,
            "CidrIpv4": "0.0.0.0/0",
            "SecurityGroupRuleArn": "arn:aws:ec2:ap-south-1:193086215187:security-group-rule/sgr-04e8379a4d332b308"
        }
    ]
}
```

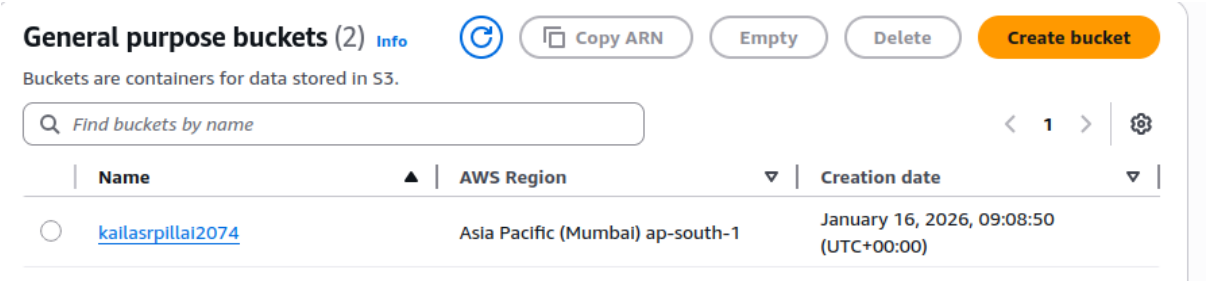
- Launch EC2 instance with Name tag and a random AMI from marketplace using **aws ec2 run-instances**

```
ubuntuuser@ubuntu25:~$ aws ec2 run-instances --image-id ami-02b8269d5e85954ef --instance-type t3.micro --key-name ec2key --security-groups ec2-sg --co
unt 1 --tag-specifications 'ResourceType=instance,Tags=[{Key=name,Value=my-ec2}]'
{
    "ReservationId": "r-0ab9f9412de32ec19",
    "OwnerId": "193086215187",
    "Groups": [],
    "Instances": [
        {
            "Architecture": "x86_64",
            "BlockDeviceMappings": [],
            "ClientToken": "b79b48c2-4f3a-4fc4-a031-a4d670c6a51a",
            "EbsOptimized": false,
            "EnaSupport": true,
            "Hypervisor": "xen",
            "NetworkInterfaces": [
                {
                    "Attachment": {
                        "AttachTime": "2026-01-16T09:02:58+00:00",
                        "AttachmentId": "eni-attach-075c244e86644fb07",
                        "DeleteOnTermination": true,
                        "DeviceIndex": 0,
                        "Status": "attaching",
                        "NetworkCardIndex": 0
                    },
                    "Description": "",
                    "Groups": [
                        {
                            "GroupId": "sg-0a345f5eacb9b2644",
                            "GroupName": "ec2-sg"
                        }
                    ],
                    "Ipv6Addresses": [],
                    "MacAddress": "0a:75:62:fa:66:f7",
                    "NetworkInterfaceId": "eni-0bc960331ad031a8d",
                    "OwnerId": "193086215187",
                    "PrivateDnsName": "ip-172-31-5-156.ap-south-1.compute.internal",
                    "PrivateIpAddress": "172.31.5.156",
```

- To terminate instance: aws ec2 terminate-instances --instance-ids <instance-ID>

- **Create S3 using AWS CLI**
    - S3 bucket names must be globally unique.
    - Created S3 bucket using aws s3api create-bucket

```
ubuntuuser@ubuntu25:~$ aws s3api create-bucket --bucket kailasrpillai2074 --region ap-south-1 --profile kailas --create-bucket-configuration LocationC
onstraint=ap-south-1
{
    "Location": "http://kailasrpillai2074.s3.amazonaws.com/"
}
ubuntuuser@ubuntu25:~$
```



    - To get the list of buckets: aws s3 ls
    - To upload files: aws s3 cp test.txt s3://my-unique-bucket-name-123/
    - To download file from S3: aws s3 cp s3://my-unique-bucket-name-123/test.txt .
    - To delete all objects inside a bucket: aws s3 rm s3://my-unique-bucket-name-123 --recursive
    - To delete bucket: aws s3api delete-bucket --bucket my-unique-bucket-name-123

Day 4 - 15/01/2026

Tasks Completed:

- **Task1:** Learned cost optimization methods
- **Task2:** Learned ways to secure aws services and aws accounts

Learnings:

- **Securing the AWS Account**
    - The AWS root account has full access to all resources.
        - Enable MFA (Multi-Factor Authentication) on the root account
        - Do not use root account for daily activities
        - Store root credentials securely
        - Create an IAM admin user and use that instead
    - IAM controls who can access what in AWS
        - IAM user: Create individual users. Assign least privilege permissions
        - IAM Groups: Group users based on roles. Attach policies to groups instead of users

        - IAM Roles: Use roles for EC2, ECS, Lambda instead of access keys. Enable cross-account access securely
    - VPC provides network isolation
        - Use private subnets for databases and internal services
        - Use public subnets only for ALB or Bastion hosts (a secure gateway server used to access private AWS resources.)
        - Restrict traffic using routing tables
    - Security Groups act as virtual firewalls
        - Allow minimum required ports (e.g., 80, 443, 22)
    - Use Bastion Host or VPN (Pritunl, AWS VPN) for SSH access
        - Avoid direct SSH access from the internet
    - Encrypt stored data for services like EBS, S3, RDS, DynamoDB
    - Cloudwatch
        - Monitor logs and metrics
        - Create alarms for unusual behavior
    - Cost and Security Optimization
        - Delete unused resources
        - Rotate access keys regularly
        - Use budgets & alerts
        - Review IAM permissions periodically
    - AWS Secrets Manager
        - Store DB passwords, API keys
        - Rotate secrets automatically
    - Use HTTPS only
    - Attach ACM certificates

Day 5 - 16/01/2026

Tasks Completed:

- **Task1:** Familiarized High Availability, Disaster Recovery, horizontal scaling, vertical scaling and resilience

Learnings:

- **High Availability (HA)**
    - High Availability is about keeping your system up and running with minimal downtime, even if something fails.
    - Multiple servers instead of one
    - Traffic is distributed
    - If one server fails then another takes over automatically
    - Examples:

        - ALB + EC2 in multiple Availability Zones
        - RDS Multi-AZ
        - Auto Scaling Groups (Set threshold to add or remove resource if more or less than threshold value)
- Disaster Recovery (DR)
    - Disaster Recovery (DR) is the process of restoring applications, systems, and data after a major failure or disaster so the business can continue operating.
    - Example
        - Daily RDS backups stored in S3
        - Cross-region replication (automatically copying data from one AWS Region to another Region.)
        - AMIs of EC2 stored as snapshots
        - Infrastructure defined using Terraform
        - If primary region fails, launch EC2 instances in DR region
- Horizontal Scaling (Scale Out / Scale In)
    - Add or remove machines to handle load.
    - Auto Scaling adds instances during traffic spikes
    - Services used are Auto Scaling Group, Lambda (automatic horizontal scaling)
- Vertical Scaling (Scale Up / Scale Down)
    - Increase or decrease the power of existing machine instead of adding more replicas.
    - Resize EC2 instance
    - Increase RDS instance class
- Resilience
    - Resilience is the ability of a system to withstand failures, recover quickly, and continue operating with little or no impact to users.
        - Redundancy: Have multiple copies of components.+
        - Fault Isolation: Failure in one component should not affect others.
        - Automatic Failover: System automatically switches to a healthy component.(Load Balancer redirects traffic, Route 53 health checks)
        - Auto Healing: System detects failure and fixes itself.System detects failure and fixes itself.
        - Scalability: System handles load without crashing.Auto Scaling Groups are Lambda auto-scaling common services that help in scalability.
        - Monitoring: Detect problems early. (Cloudwatch)
        - Backup: Recover from major failures. (Snapshots, CRR)